Лабораторна робота 6: Міграції схем за допомогою Flyway

Контекст

У попередніх лабораторних роботах (Лабораторні роботи 1–5) ви розробили та нормалізували схему бази даних PostgreSQL. У цій роботі (Лабораторна робота 6) ви навчитеся поступово змінювати цю схему за допомогою Flyway – інструмента керування міграціями баз даних. Flyway – це open-source засіб для впорядкування версійних змін схеми: він застосовує SQL-скрипти з фіксованими номерами версій, забезпечуючи коректне оновлення бази. Наприклад, Flyway підтримує схеми багатьох СУБД (у тому числі PostgreSQL) і зберігає виконані міграції у власній системній таблиці flyway_schema_history, щоб знати, які скрипти вже застосовані. Це забезпечує відтворюваність та цілісність змін схеми.

Цілі

- Використати Flyway для керування схемами та дослідити, як Flyway може аналізувати та змінювати схему вашої бази даних.
- Зрозуміти конвенцію іменування Flyway-скриптів, застосування міграцій, генерування та застосування змін схеми.
- Написати кілька версійних SQL-міграцій для вашої схеми та застосувати їх через Flyway.
- Перевірити результати змін за допомогою SQL-запитів і задокументувати їх.
- Навчитися коректно використовувати контролювання версій міграцій у Git (скрипти зберігаються у проекті, а не змінюються після застосування).

Інструкції

1. Створіть новий Java/Maven проєкт. Наприклад, у консолі запустіть `mvn archetype:generate` або скористайтесь IDE (IntelliJ IDEA чи Eclipse) для створення простого Java-проекту.
2. Налаштуйте підключення до БД. У `pom.xml` вкажіть параметри з'єднання (наприклад `<url>jdbc:postgresql://localhost:5432/yourdb</url>`, `<user>`, `<password>` в конфігурації плагіну), або створіть файл `application.properties` з `spring.datasource.url`, `username`, `password`, якщо ви використовуєте Spring Boot. Важливо, щоб Flyway мав доступ до вашої локальної бази з нормалізованою схемою.
3. Інтроспекція (опціонально). Якщо хочете, можете перевірити поточну схему з нормалізованими таблицями (з Лабораторної роботи 5). Наприклад, виконайте у плагіні Flyway команду типу `mvn flyway:info` (якщо плагін налаштовано) або `flyway info`, щоб побачити поточні версії (має бути версія 1 або 0 із пустим schema). Це допоможе пересвідчитись, що нічого не застосовано раніше.

4. Написання міграцій. Створіть щонайменше **3** нові міграційні скрипти для зміни схеми. У проєкті створіть папку **`src/main/resources/db/migration/`** (якщо ви налаштували Spring Boot або відповідно `flyway.locations`), або `src/main/resources/db/migration`. Усі SQL-файли, які ви туди покладете, будуть застосовані Flyway. Назви файлів повинні відповідати шаблону: `V{номер}__{опис}.sql`. Наприклад:

   `V1__create_reviews_table.sql`: створити нову таблицю, наприклад `reviews` чи іншу, згідно з логікою вашої теми (наприклад, для онлайн-магазину або університету).

   `V2__add_column_to_products.sql`: додати новий стовпець у наявну таблицю (наприклад, додати поле `rating` у `Product` чи `Student`).

   `V3__drop_unused_column.sql`: видалити непотрібний стовпець (наприклад, якщо ви прибрали колонку або перемістили дані під час нормалізації). У тілі кожного скрипта використовуйте звичайний SQL DDL (наприклад, `CREATE TABLE`, `ALTER TABLE`, `DROP COLUMN`). Кожен скрипт має бути *істотною* одиницею змін: після застосування буде змінено схему (додавання/зміна таблиці або стовпця). Пам'ятайте, що версійовані міграції не повинні змінюватися після запуску – для додаткових правок створюйте нові міграції.

5. Після створення скриптів виконайте їх у вашу базу за допомогою Flyway.

6. Якщо ви використовуєте Maven-плагін, запустіть команду (у консолі, у корені проекту): `mvn flyway:migrate`. Якщо Gradle – `gradle flywayMigrate`. Flyway сам під'єднається до вказаної бази та застосує всі нові міграції у правильному порядку.
7. Після виконання повинен з'явитись звіт у консолі: всі ваші `V1__...`, `V2__...` скрипти матимуть статус `SUCCESS`. Також у базі буде створено таблицю `flyway_schema_history`, де Flyway позначає виконані скрипти. Переконайтеся, що помилок немає, і всі зміни відображаються (наприклад, через перевірку схеми в pgAdmin або DBeaver).

8. Відкрийте вашу базу (pgAdmin, DBeaver або psql) і перевірте, що зміни застосовано.

   - Перегляньте структуру таблиць (`\d table_name` або правим кліком *View Data > View Columns* у pgAdmin). Створені таблиці та нові стовпці мають з'явитися.
   - Запустіть SELECT-запити, щоб побачити дані. Наприклад, `SELECT * FROM reviews;` чи `SELECT * FROM products;` та перевірте, що нові поля (які Ви додали) можуть містити значення (або мають початкові значення за замовчуванням).
   - Збережіть результати цих перевірок: це можуть бути скріншоти виводу або текстовий лог команд psql. Вам потрібно буде подати ці результати.

9. Документація і Git. Створіть файл `migration-notes.md` або доповніть README. У ньому опишіть кожну міграцію: що було змінено у схемі і навіщо (наприклад, «У міграції V2 додано стовпець `in_stock` у таблицю `Product`, щоб відзначати наявність товару»). Вкажіть номери версій і короткий опис дій. Також прикладіть важливі фрагменти результатів SELECT-запитів або посилання на скріншоти, які підтверджують застосування змін.

10. **Важливо.** Закомітьте у ваш репозиторій **усі** файли проекту: `pom.xml` (із налаштуваннями Flyway), папку `src/main/resources/db/migration/` з вашими SQL-скриптами, а також документ `migration-notes.md` і, за потреби, файли/зображення з результатами запитів. Після цього зафіксуйте комміти в Git.

Поради

- Використовуйте **контроль версій**: кожна зміна схеми оформляється новим SQL-скриптом з унікальним номером.
- Перевіряйте міграції на копії бази (можна створити нову базу або резервну копію) перед застосуванням у продакшн. Flyway дозволяє безпечно повторювати перенесення при необхідності (повторювані скрипти ураховуються за контрольними сумами).
- **Не змінюйте** вже застосовані скрипти! Якщо потрібно виправити щось, створіть новий скрипт з наступним номером. Flyway не дозволяє повторно застосувати редагований скрипт.
- Можна використовувати **Java API Flyway** (наприклад, через виклик `Flyway.configure().dataSource(...).load().migrate()` у коді), але для цієї роботи достатньо Maven/Gradle-плагіну.

Результати

- - Міграційні скрипти у папці `src/main/resources/db/migration/` (файли `V1__...`, `V2__...` тощо).
- Оновлений `pom.xml` із зазначеними залежностями/плагінами Flyway.
- запитів.
- Результати SELECT-запитів (текстом або скріншотами) для доказу того, що скрипти застосовано.
- **Звіт Markdown** migration-notes.md у вашому репозиторії. Для кожної міграції поясніть, що змінилося, та покажіть фрагменти коду «до»/«після». Додайте будь-які запити та знімки екрана, що підтверджують результат. Дозволяється звіт у форматах pdf/docx.

How to Use Flyway for Database Migrations in Spring Boot Applications | The IntelliJ IDEA Blog

https://blog.jetbrains.com/idea/2024/11/how-to-use-flyway-for-database-migrations-in-spring-boot-applications/

Database Schema Migration Made Simple

https://nextnative.dev/blog/database-schema-migration

Database Migrations with Flyway | Baeldung

https://www.baeldung.com/database-migrations-with-flyway